

Spécification fonctionnelle et technique

Assistant vocal distant pour Claude Code dans VS Code

Version 1.0 — 19 août 2026

Objectif. Construire une interface web mobile permettant de dialoguer vocalement avec une instance de Claude Code exécutée sur le PC, sans devoir manipuler le clavier ni le téléphone pendant la conversation.

1. Vision du produit

Le téléphone sert d'interface vocale. Le PC reste le moteur d'exécution : il reçoit le message, le retranscrit si nécessaire, le transmet à Claude Code dans le contexte du projet VS Code, récupère une réponse courte, la convertit en audio et la renvoie au téléphone.

L'expérience cible est proche d'un chat vocal : parler → détection automatique de fin de phrase → envoi → traitement sur PC → réponse vocale → reprise automatique de l'écoute.

2. Architecture recommandée

Composant	Technologie recommandée	Rôle
Téléphone	PWA HTML/CSS/JS	UI, microphone, affichage, lecture audio
Transport	WebSocket	Canal bidirectionnel temps réel téléphone ↔ PC
Serveur PC	Node.js / TypeScript	Session, orchestration, sécurité, files d'attente
STT	Web Speech API en MVP ou moteur local Whisper	Reconnaissance vocale
Agent de code	Claude Code	Exécution dans le projet VS Code
TTS	SpeechSynthesis côté mobile ou Piper TTS sur PC	Génération de voix
Accès réseau	Wi-Fi local / Tailscale en option	Connexion téléphone ↔ PC

Choix important : WebSocket

WebSocket est préférable à un simple polling HTTP : le téléphone peut envoyer un message et recevoir immédiatement les états, le texte de réponse, les erreurs et l'audio. Une seule connexion persistante simplifie également la gestion de la conversation.

3. Flux utilisateur cible

1. L'utilisateur ouvre la PWA sur son téléphone.
2. La PWA établit une connexion WebSocket sécurisée avec le serveur du PC.
3. L'application passe en mode écoute.
4. L'utilisateur parle naturellement.
5. Le système détecte la fin du message à partir de l'absence de parole et/ou du résultat final du moteur STT.
6. Le texte est envoyé au PC.
7. Le serveur injecte le message dans Claude Code avec le contexte du projet actif.
8. Claude Code produit une réponse volontairement synthétique.
9. Le serveur convertit cette réponse en audio.
10. L'audio est envoyé au téléphone et joué automatiquement.

11. À la fin de la lecture, l'application repasse automatiquement en écoute.

4. Détection automatique de fin de message

Le système ne doit pas attendre un bouton « Envoyer ». La fin du message doit être déterminée automatiquement.

MVP recommandé : utiliser la reconnaissance vocale du navigateur lorsque disponible, avec un délai d'inactivité court après le dernier résultat final. L'API Web Speech expose notamment les résultats finaux et les événements de fin de reconnaissance. Son support reste toutefois limité selon les navigateurs. [\[cite\]turn0search0\[turn0search8\]turn0search9\[](#)

Version robuste : capturer le flux audio avec MediaRecorder/Web Audio et utiliser un VAD (Voice Activity Detection) local pour déterminer précisément le début et la fin de parole. Le STT peut alors être exécuté localement avec Whisper ou une implémentation équivalente.

Paramètres initiaux suggérés : début de parole après environ 150–300 ms d'activité ; fin de parole après environ 700–1200 ms de silence ; durée maximale configurable par message.

5. Stratégie gratuite

Pour minimiser les coûts récurrents, privilégier les composants locaux et open source.

Besoin	MVP gratuit	Version robuste
STT	Web Speech API	Whisper local
TTS	SpeechSynthesis du téléphone	Piper TTS local sur PC
Transport	WebSocket sur Wi-Fi	WebSocket + Tailscale si accès hors LAN
Backend	Node.js / TypeScript	Node.js / TypeScript
Agent	Claude Code existant	Même intégration

La Web Speech API permet la reconnaissance et la synthèse vocales dans une application web ; certaines implémentations peuvent fonctionner localement avec un pack de langue, mais la disponibilité dépend du navigateur. [\[cite\]turn0search11\[turn0search6\[](#)

Pour une solution entièrement locale, Piper constitue une option de TTS rapide et locale ; des voix françaises existent dans l'écosystème Piper. [\[cite\]turn0search12\[turn0search14\[](#)

6. Intégration Claude Code

Le serveur PC doit être un orchestrateur autour de Claude Code et non une seconde IA.

Principe : une session est associée à un projet/répertoire de travail. Chaque message vocal est transmis à la même session Claude Code afin de conserver le contexte.

La réponse destinée à la voix doit être contrainte par un prompt système du type : « Réponds de façon extrêmement synthétique. Maximum 2 à 4 phrases. Pas de markdown inutile. Si une action est nécessaire, indique uniquement l'action et son résultat. »

L'intégration exacte avec Claude Code doit être réalisée à partir de l'interface officiellement disponible dans la version installée, plutôt que par une simulation fragile de frappes clavier dans VS Code.

7. Protocole WebSocket

Messages JSON recommandés :

```
Client → PC : session.start avec l'identifiant de session et le projet demandé.
Client → PC : user.message avec le texte transcrit.
PC → Client : state avec les états listening, processing, speaking, error.
```

PC → Client : assistant.text avec la réponse textuelle.
PC → Client : assistant.audio avec l'audio ou une URL temporaire.
PC → Client : error avec un code et un message court.

8. États de l'application mobile

État	Comportement
CONNECTING	Connexion au serveur PC.
LISTENING	Microphone actif, attente de parole.
TRANSCRIBING	Message en cours de finalisation.
PROCESSING	Claude Code traite la demande.
SPEAKING	Lecture de la réponse audio.
ERROR	Affichage d'une erreur courte et tentative de reconnexion.

9. UI mobile

Interface volontairement minimale :

- Un indicateur de connexion PC.
- Un grand indicateur circulaire du mode actuel : écoute / traitement / réponse.
- Le dernier message utilisateur affiché en petit.
- La dernière réponse de Claude affichée en texte.
- Un bouton d'arrêt d'urgence du microphone.
- Aucun bouton « envoyer » dans le flux normal.

10. Sécurité

- Ne pas exposer directement le serveur WebSocket sur Internet sans authentification.
- Pour le LAN, utiliser un token aléatoire de session ou un secret partagé.
- Pour l'accès depuis l'extérieur du domicile, préférer un réseau privé type Tailscale plutôt qu'un port public.
- Limiter les projets/répertoires auxquels le serveur peut demander à Claude Code d'accéder.
- Ne jamais envoyer les clés ou secrets du PC dans l'interface mobile.

11. Gestion des interruptions

L'utilisateur doit pouvoir interrompre une réponse vocale. Si une nouvelle parole est détectée pendant la lecture, le client peut arrêter immédiatement l'audio et passer en écoute. Le serveur doit également pouvoir annuler le traitement Claude Code lorsque cela est techniquement supporté.

Une file d'attente de messages doit être évitée par défaut : le système doit fonctionner en mode conversationnel « un message utilisateur → une réponse ».

12. Critères d'acceptation MVP

- ☐ Depuis un téléphone connecté au même Wi-Fi, ouvrir la PWA et établir automatiquement la connexion au PC.
- ☐ Parler sans appuyer sur « envoyer ».

- ☐ Détecter automatiquement la fin du message.
- ☐ Recevoir le texte sur le PC.
- ☐ Transmettre le texte à Claude Code dans le bon projet.
- ☐ Recevoir une réponse courte.
- ☐ Transformer la réponse en audio.
- ☐ Lire automatiquement l'audio sur le téléphone.
- ☐ Repasser automatiquement en écoute après la réponse.
- ☐ Pouvoir interrompre la lecture.
- ☐ Reprendre automatiquement après une erreur réseau simple.

13. Roadmap d'implémentation

Phase 1 — Proof of concept : PWA + WebSocket + texte uniquement.

Phase 2 — Voix : microphone mobile + STT + détection de silence.

Phase 3 — Claude Code : branchement de la session Claude Code au serveur PC.

Phase 4 — Réponse vocale : TTS et lecture automatique.

Phase 5 — Conversation : interruption, reprise automatique, états UI et reconnexion.

Phase 6 — Robustesse : VAD local, Whisper local, Piper local, authentification et accès distant.

14. Recommandation finale

Le meilleur rapport simplicité / coût / qualité pour démarrer est : PWA mobile + WebSocket + Node.js/TypeScript sur le PC + Web Speech API pour le MVP + Claude Code + SpeechSynthesis sur le téléphone. Cela permet de valider très rapidement toute la boucle conversationnelle.

Une fois le flux validé, remplacer progressivement les briques dépendantes du navigateur par VAD + Whisper local pour le STT et Piper local pour le TTS. Cette architecture réduit la dépendance aux services cloud et permet de conserver l'audio et les transcriptions sur le réseau local. La Web Speech API reste une excellente option de prototypage, mais sa compatibilité navigateur n'est pas suffisamment uniforme pour constituer à elle seule le socle d'une version robuste.

`[cite]turn0search0[turn0search11]`

15. Arborescence de projet proposée

```
voice-code-bridge/
├── server/
│   ├── websocket.ts
│   ├── session.ts
│   ├── claude-code.ts
│   ├── tts.ts
│   └── security.ts
├── mobile/
│   ├── index.html
│   ├── app.js
│   ├── audio.js
│   └── styles.css
├── shared/
│   └── protocol.ts
├── config/
│   └── example.env
└── README.md
```

16. Décisions à prendre avant développement

- Système d'exploitation du PC : Windows, macOS ou Linux.
- Téléphone : iPhone, Android ou les deux.
- Connexion uniquement sur le Wi-Fi local ou également depuis l'extérieur.
- STT cloud temporaire ou STT entièrement local dès le départ.
- TTS navigateur ou Piper local.
- Méthode exacte d'intégration avec la version de Claude Code installée.